VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING

—o0o—

PROJECT2 REPORT

Sorting base-on FPGA

SUPERVISOR: Nguyễn Trung Hiếu
STUDENT: Nguyễn Đại Đồng
ID: 2210780

Ho Chi Minh, ../../20..

Mục lục

Abstract **iii**

Acknowledgement **iv**

1 Introduction **1**
1.1 Đặt vấn đề 1
1.2 Thách thức trong thiết kế phần cứng 1
1.2.1 Thách thức của Kiến trúc Máy tính Truyền thống và Hạn chế của Thuật toán Hiện hành 1
1.2.2 Đề xuất Giải pháp Kiến trúc Processing-In-Memory (PIM) 2
1.3 Mục tiêu của báo cáo 3

2 Literature Review **4**
2.1 Thực hiện lại Framework của bài báo 5

3 System Level Test **12**

4 RTL Implementation **13**
4.1 Bộ thực hiện tính giá trị trung bình của mảng (PACM_UNIT) 13
4.1.1 Bộ tính cộng số thực (BFP16_add) 13
4.1.2 Bộ tính cộng tích lũy của mảng (PA_Cal_Sum) 14
4.1.3 Bộ tính chia số thực (FP32_div) 17
4.2 Bộ thực hiện partition lại mảng theo giá trị trung bình (PAMEM_UNIT) 17
4.3 Bộ thực hiện đệ quy cho giải thuật (DI_UNIT) 17

5 Verification **18**

References **19**

Danh sách hình vẽ

1 Bộ cộng số thực. 13
2 Bộ điều khiển dữ liệu đọc với Memory Unit. 14
3 Bộ phát hiện tính hiệu bắt đầu. 15
4 Bộ tính cộng số thực (BFP16_add). 16
5 Bộ chia số thực. 17

Danh sách bảng

1 The time complexity of the conventional and proposed Quick and Merge sorts in the best, average, and worst cases. 6

List of Listings

1 The Pseudo-Code of the Serial Realization of the Proposed Mean-Based Data Sorting Framework. 6
2 Kết quả kiểm chứng khi chạy với $N = 2^{17}$. 7
3 Kiểm tra sử dụng bộ nhớ khi sử dụng QuickSort. 8
4 Kiểm tra sử dụng bộ nhớ khi sử dụng QuickSort kèm theo Framework. . 8
5 Kiểm tra sử dụng bộ nhớ khi sử dụng MergeSort. 9
6 Kiểm tra sử dụng bộ nhớ khi sử dụng MergeSort kèm theo Framework. . 10

Abstract

Đồ án thực hiện một bộ có thể cho phép phần cứng sắp xếp dữ liệu trên một mảng lớn (tầm GB) mà không cần sử dụng đến CPU, giúp giải quyết được vấn đề liên quan việc tổn hao trên bus khi sử dụng CPU giao tiếp với Memory External.

Acknowledgement

Xin cảm ơn thầy Nguyễn Trung Hiếu.

Chapter 1. Introduction

1.1 Đặt vấn đề

Trong kỷ nguyên số hóa hiện nay, sự bùng nổ của Dữ liệu lớn (Big Data), Internet of Think (IoT) và Trí tuệ nhân tạo (AI) đã dẫn đến sự gia tăng theo cấp số nhân về khối lượng dữ liệu cần lưu trữ và xử lý. Sắp xếp dữ liệu (Data Sorting) không chỉ đơn thuần là việc tổ chức lại thông tin mà còn là một kernel tính toán nền tảng (fundamental computational kernel) quyết định hiệu năng của toàn bộ hệ thống. Sắp xếp đóng vai trò tiền đề thiết yếu cho các thuật toán tìm kiếm (như Binary Search), nén dữ liệu, loại bỏ trùng lặp và tối ưu hóa truy vấn cơ sở dữ liệu.

Các lĩnh vực yêu cầu lưu trữ và xử lý lượng dữ liệu khổng lồ bao gồm:

- Hệ quản trị cơ sở dữ liệu (Database Management Systems): Sắp xếp là bước bắt buộc để tạo chỉ mục (indexing), giúp tăng tốc độ truy vấn SQL và tối ưu hóa các phép toán JOIN.
- Trí tuệ nhân tạo và Học máy (AI/ML): Các thuật toán như k-Nearest Neighbors (k-NN), lọc trung vị (median filtering) trong xử lý ảnh, và tiền xử lý dữ liệu huấn luyện đều phụ thuộc nặng nề vào tốc độ sắp xếp.
- Viễn thông và Mạng (5G/6G & IoT): Trong các hệ thống MIMO quy mô lớn hoặc định tuyến gói tin, việc sắp xếp dữ liệu tín hiệu và ưu tiên luồng dữ liệu (traffic shaping) đòi hỏi độ trễ cực thấp.
- Tin sinh học (Bioinformatics): Sắp xếp các chuỗi gen và protein để tìm kiếm mẫu tương đồng trong các cơ sở dữ liệu sinh học khổng lồ.

1.2 Thách thức trong thiết kế phần cứng

1.2.1 Thách thức của Kiến trúc Máy tính Truyền thống và Hạn chế của Thuật toán Hiện hành

a. Giới hạn của Kiến trúc Von Neumann và Bức tường Bộ nhớ

Trong bối cảnh bùng nổ của các ứng dụng thâm dụng dữ liệu (Data-intensive applications) như Big Data và IoT, kiến trúc máy tính truyền thống Von Neumann đang đối mặt với những giới hạn vật lý nghiêm trọng. Sự chênh lệch ngày càng lớn giữa tốc độ xử lý của CPU và tốc độ truy xuất bộ nhớ đã tạo ra hiện tượng *"Bức tường bộ nhớ"* (Memory Wall). Việc di chuyển khối lượng lớn dữ liệu chưa được

sắp xếp giữa bộ nhớ ngoại vi (Off-chip DRAM) và bộ vi xử lý trung tâm (Host CPU) thông qua Bus hệ thống dẫn đến hai vấn đề cốt lõi trong thiết kế vi mạch:

- **Độ trễ truy xuất (Access Latency):** Thời gian chờ dữ liệu từ DRAM làm giảm hiệu suất thực thi của pipeline xử lý.
- **Tiêu thụ năng lượng động (Dynamic Power Consumption):** Việc chuyển đổi trạng thái liên tục trên Bus dữ liệu gây hao phí năng lượng đáng kể, đi ngược lại các tiêu chuẩn về thiết kế công suất thấp (Low-power design).

b. Hạn chế của Thuật toán Sắp xếp Kinh điển trong Triển khai Phần cứng

Mặc dù các thuật toán như Merge-sort và Quick-sort hoạt động hiệu quả trên phần mềm, việc ánh xạ chúng lên kiến trúc phần cứng song song (Parallel Hardware Architecture) gặp nhiều trở ngại:

- **Merge-sort:** Tồn tại tính chất tuần tự cao trong giai đoạn hợp nhất (Merge phase), tạo ra các phụ thuộc dữ liệu (Data dependency) khiến việc thiết kế các bộ xử lý song song hoàn toàn trở nên bất khả thi mà không tiêu tốn tài nguyên bộ nhớ đệm (Buffer memory) lớn.
- **Quick-sort:** Việc lựa chọn phần tử chốt (Pivot) ngẫu nhiên thường dẫn đến việc phân hoạch không đồng đều, tạo ra các mảng con mất cân bằng (Unbalanced subarrays). Trong phần cứng, điều này gây ra hiện tượng *mất cân bằng tải* (Load Imbalance) giữa các đơn vị xử lý (Processing Units - PUs), khiến một số lõi hoàn thành sớm và rơi vào trạng thái rỗi (Idle state), làm giảm hiệu suất sử dụng tài nguyên (Resource Utilization).

1.2.2 Đề xuất Giải pháp Kiến trúc Processing-In-Memory (PIM)

a. Chuyển dịch sang Xử lý Tại Bộ nhớ (Near-Memory Processing)

Để giải quyết nút thắt cổ chai về băng thông và độ trễ, đồ án đề xuất thiết kế một lõi IP (Intellectual Property Core) hoạt động theo cơ chế *Processing-In-Memory* (PIM). Kiến trúc này cho phép thực thi tác vụ sắp xếp ngay tại miền bộ nhớ, giảm tải (Offload) hoàn toàn cho Host CPU và giảm thiểu lưu lượng giao tiếp trên Bus (Bus Traffic Reduction).

b. Kỹ thuật Phân hoạch Dựa trên Giá trị Trung bình (Mean-based Partitioning)

Thiết kế phần cứng đề xuất áp dụng phương pháp phân hoạch dữ liệu dựa trên giá trị trung bình để chia tập dữ liệu lớn thành các mảng con độc lập có kích thước xấp xỉ bằng nhau (Independent subarrays of approximately equal length). Phương pháp này mang lại các ưu điểm vượt trội cho thiết kế IC:

- **Cân bằng tải (Load Balancing):** Đảm bảo khối lượng tính toán được phân phối đều giữa các lõi xử lý song song, tối đa hóa thông lượng (Throughput).
- **Kiến trúc Tự chủ (CPU-less Architecture):** Các mảng con độc lập cho phép các khối phần cứng hoạt động song song mà không cần cơ chế đồng bộ hóa phức tạp hay sự điều phối từ CPU.
- **Tối ưu hóa Năng lượng (Energy Efficiency):** Việc thực hiện sắp xếp tại chỗ (In-place sorting) và loại bỏ các thao tác I/O dư thừa giúp cải thiện đáng kể chỉ số hiệu năng trên mỗi watt (Performance-per-Watt).

1.3 Mục tiêu của báo cáo

Mục tiêu của đồ án là thiết kế một kiến trúc **Processing-In-Memory (PIM)** đóng vai trò như một bộ tăng tốc phần cứng (Hardware Accelerator), cho phép sắp xếp các tập dữ liệu kích thước lớn lưu trữ tại bộ nhớ ngoài (Off-chip Memory/DRAM) mà giảm thiểu tối đa sự phụ thuộc vào vi xử lý trung tâm (Host CPU).

Khối thiết kế này ứng dụng kỹ thuật phân hoạch dữ liệu thành các mảng con độc lập (Independent Subarrays) và thực hiện sắp xếp tại chỗ (In-place). Cơ chế này giúp loại bỏ nhu cầu di chuyển dữ liệu liên tục qua Bus hệ thống, từ đó giải quyết triệt để vấn đề nút thắt băng thông (Memory Wall) và tiêu hao năng lượng do truyền tải dữ liệu. Bằng cách giảm tải (offload) tác vụ sắp xếp khỏi CPU, thiết kế giúp tối ưu hóa hiệu năng tổng thể và hiệu quả năng lượng cho các ứng dụng xử lý dữ liệu lớn (Big Data applications).

Chapter 2. Literature Review

Nghiên cứu này được xây dựng dựa trên nền tảng lý thuyết của framework sắp xếp dữ liệu đề xuất bởi Shahriar Shirvani Moghaddam và Kiaksar Shirvani Moghaddam trong công trình *"A General Framework for Sorting Large Data Sets Using Independent Subarrays of Approximately Equal Length"* [1]. Khác với các phương pháp tiếp cận truyền thống thường tập trung vào việc tối ưu hóa một thuật toán cụ thể, framework này cung cấp một cơ chế tổng quát để chia nhỏ bài toán sắp xếp lớn thành các tác vụ độc lập, tạo tiền đề cho việc xử lý song song hiệu quả.

Cơ chế hoạt động của framework được chia thành ba giai đoạn chính (phases), được thiết kế để khắc phục nhược điểm mất cân bằng tải thường thấy ở Quick-sort truyền thống:

Phase 1. Sorted/Similar Checker - SS Checker

Trước khi thực hiện bất kỳ thao tác phân hoạch nào, framework thực hiện một bước kiểm tra trạng thái dữ liệu đầu vào. Khối *SS Checker* có nhiệm vụ xác định xem mảng dữ liệu hiện tại có thuộc một trong các trường hợp đặc biệt sau hay không:

- Đã được sắp xếp hoàn toàn (Sorted).
- Được sắp xếp theo thứ tự ngược (Reversely sorted).
- Chứa các phần tử giống hệt nhau (Similar elements).

Việc phát hiện sớm các trường hợp này giúp hệ thống bỏ qua các bước tính toán dư thừa, từ đó giảm thiểu độ phức tạp thời gian trong các trường hợp tốt nhất (Best-case scenario).

Phase 2. Mean-based Partitioning

Đây là giai đoạn cốt lõi tạo nên tính ưu việt của giải thuật đối với thiết kế phần cứng. Thay vì chọn phần tử chốt (pivot) ngẫu nhiên như Quick-sort, framework tính toán giá trị trung bình (Mean value) của toàn bộ mảng dữ liệu.

- Dữ liệu được so sánh với giá trị Mean và chia thành hai mảng con: một mảng chứa các giá trị nhỏ hơn Mean và mảng còn lại chứa các giá trị lớn hơn hoặc bằng Mean.
- Quá trình này được thực hiện đệ quy qua M cấp độ (Levels), tạo ra 2^M mảng con.

- **Đặc tính quan trọng:** Các tác giả đã chứng minh rằng việc sử dụng Mean làm pivot giúp tạo ra các mảng con có độ dài xấp xỉ bằng nhau (Approximately equal length) ngay cả với các phân phối dữ liệu bất đối xứng. Điều này đảm bảo tính cân bằng tải (Load balancing) tuyệt đối khi ánh xạ lên các phần tử xử lý song song.

Phase 3. Core-Sort

Sau khi đạt đến số lượng phân hoạch M định trước, các mảng con độc lập (Independent Subarrays) sẽ được sắp xếp riêng biệt bằng một thuật toán lõi (Core-sort) như Quick-sort hoặc Merge-sort.

Do các mảng con đã được phân tách hoàn toàn về mặt giá trị (mọi phần tử ở mảng con i đều nhỏ hơn mảng con $i + 1$), quá trình này có thể thực hiện song song hoàn toàn trên các lõi xử lý phần cứng mà không cần cơ chế đồng bộ hóa hay trao đổi dữ liệu giữa các luồng (Inter-processor communication).

Tổng hợp lại, framework này cung cấp một kiến trúc lý tưởng cho thiết kế *Processing-In-Memory* (PIM) hướng tới mục tiêu CPU-less, vì nó loại bỏ sự phụ thuộc dữ liệu và cho phép các khối logic tính toán hoạt động định hình và cô lập.

2.1 Thực hiện lại Framework của bài báo

Trong bài báo nêu ra được cải thiện độ phức tạp khi sử dụng framework trong việc sử dụng thuật toán QuickSort và MergeSort như sau:

Bảng 1: The time complexity of the conventional and proposed Quick and Merge sorts in the best, average, and worst cases.

	Method	**Best-case**	**Average-case**	**Worst-case**
Quick	Conventional	$O(N \log N)$	$O(N \log N)$	$O(N^2)$
	Proposed serial	$\begin{cases} O(2N), & \text{ord./sim.} \\ O(3N), & \text{rev. ord.} \end{cases}$	$O(4M.N) + O(N \log(\frac{N}{2^M}))$	$O(4M.N) + O((2^M - k)\frac{N}{2^M} \log(\frac{N}{2^M}) + k\frac{N^2}{2^{2M}})$
	Proposed parallel	$\begin{cases} O(\frac{2N}{2^M} + 2M), & \text{ord./sim.} \\ O(\frac{3N}{2^M} + 3M), & \text{rev. ord.} \end{cases}$	$O(4M(\frac{N}{2^M} + 2^M - 1)) + O(\frac{N}{2^M} \log(\frac{N}{2^M}))$	$O(4M(\frac{N}{2^M} + 2^M - 1)) + O(\frac{N^2}{2^{2M}})$
Merge	Conventional	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$
	Proposed serial	$\begin{cases} O(2N), & \text{ord./sim.} \\ O(3N), & \text{rev. ord.} \end{cases}$	$O(4M.N) + O(N \log(\frac{N}{2^M}))$	$O(4M.N) + O(N \log(\frac{N}{2^M}))$
	Proposed parallel	$\begin{cases} O(\frac{2N}{2^M} + 2M), & \text{ord./sim.} \\ O(\frac{3N}{2^M} + 3M), & \text{rev. ord.} \end{cases}$	$O(4M(\frac{N}{2^M} + 2^M - 1)) + O(\frac{N}{2^M} \log(\frac{N}{2^M}))$	$O(4M(\frac{N}{2^M} + 2^M - 1)) + O(\frac{N}{2^M} \log(\frac{N}{2^M}))$

Và đây là đoạn code pseudo thực hiện:

Listing 1: The Pseudo-Code of the Serial Realization of the Proposed Mean-Based Data Sorting Framework.

```
arr: Array of data;
    arr.Leangth: Length of the array;
    M: Number of divisions;
    cnt: Global counter for divisions;
    si: Starting index;
    ei: Ending index;
    iCnt: Global counter to store the index in indexes;
    subarray: 0 - unsorted, 1 - sorted,
              2 - reversely sorted, 3 - similar;
    bi: Boundary index;
    Function Sort(arr, M):
        cnt <- 0;
        si <- 0;
        Division(arr, 0, arr.Length - 1, M);
```

```
    EndFunction
EndFunction
Function Division(arr, si, ei, M):
    bool SS_check = SS_check(arr, si, ei);
    case(SS_check):
        1, 3: return;
        2: Reverse(arr, si, ei);
        0:
            if(cnt < (1<<M) - 1):
                bi <- Partition(arr, si, ei);
                cnt <- cnt + 1;
                Division(arr, si, bi - 1, M);
                if(si = 0 or ei = arr.Length - 1):
                    iCnt <- 1;
                Division(arr, bi + 1, ei, M);
            else:
                Sort form si -> ei using Core-Sort;
    return;
EndFunction
Function Partition(arr, si, ei):
    pivot <- mean(arr, si, ei)
    bi <- si - 1;
    for i = si to ei:
        if arr[j] <= pivot:
            bi <- bi + 1;
            Swap(arr[bi], arr[i]);
    return bi;
EndFunction
```

Ta tiến hành thực hiện việc so các giải thuật trên với các phần tử số thực như sau:

```
-1344457441.558848 1837028809.6233559 ... 1967142726.7494054
    421697126.41092634
```

```
Finished reading file: ./tools/unsorted.txt
Finish write file './Reports/COMPILE_REPORT/unsorted.txt'.
Size of array: 131072
Number subarry M = 5
[FrameworkStandard_Quick] Number of Similar = 0
[FrameworkStandard_Quick] Number of Ascending = 83
[FrameworkStandard_Quick] Number of Descending = 84
[FrameworkStandard_Merge] Number of Similar = 0
[FrameworkStandard_Merge] Number of Ascending = 83
[FrameworkStandard_Merge] Number of Descending = 84

Sort algorithm              Check sorted   Time (ns)          Number Swap        Number Compare
-----------------------------------------------------------------------------------------------------
Data Set                    FAIL           0                  0                  0
-----------------------------------------------------------------------------------------------------
Sort Standard               PASS           6776394            0                  0
-----------------------------------------------------------------------------------------------------
QuickSort                   PASS           8550932            2664420            1422629
-----------------------------------------------------------------------------------------------------
Framework_QuickSort         PASS           10602239           128005830          63593393
```

```
-----------------------------------------------------------------------------------------------
MergeSort                    PASS          8682661            2664420            1422629
-----------------------------------------------------------------------------------------------
Framework_Merge              PASS          14602192           370425084          134830050
-----------------------------------------------------------------------------------------------
```

Listing 2: Kết quả kiểm chứng khi chạy với $N = 2^{17}$.

```
--------------------------------------------------------------------------------
Command:            ./main 5 2
Massif arguments:   --stacks=yes --massif-out-file=Reports/ANALYSIS_DIR/checkmemory
  /QuickSort_Result.out
ms_print arguments: Reports/ANALYSIS_DIR/checkmemory/QuickSort_Result.out
--------------------------------------------------------------------------------


MB
1.071^                                                                  :::
|                                                                      #:::
|                                                                      #:::
|                                                                      #:::
|                                                                      #:::
|              @                                                       #:::
|              @                                                       #:::
|              @                                                       #:::
|              @                                                       #:::
|              @                                                       #:::
|              @::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
|              @::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
|              @::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
|              @::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
|        ::::::@::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
|        ::::::@::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
|        ::::::@::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
|    ::::::::::@::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
| ::: ::::::::@::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
|:: : ::::::::@::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
0 +----------------------------------------------------------------------->Mi
0                                                                     787.9

Number of snapshots: 96
Detailed snapshots: [13, 17, 21, 25, 52, 62, 72, 82, 90 (peak)]
```

Listing 3: Kiểm tra sử dụng bộ nhớ khi sử dụng QuickSort.

```
--------------------------------------------------------------------------------
Command:            ./main 5 2
Massif arguments:   --stacks=yes --massif-out-file=Reports/ANALYSIS_DIR/checkmemory
  /QuickSort_Convenlution_Result.out
ms_print arguments: Reports/ANALYSIS_DIR/checkmemory/QuickSort_Convenlution_Result.
  out
--------------------------------------------------------------------------------
```

```
MB
1.071^                                                                  :
|                                                                      #:::
|                                                                      #:::
|                                                                      #:::
|                                                                      #:::
|              @                                                       #:::
|              @                                                       #:::
|              @                                                       #:::
|              @                                                       #:::
|              @                                                       #:::
|              @:::::::::::::::@:@@:@:::::@::::::::::::::::::@::::@::#:::
|              @::: :::: ::::::@:@ :@:::: @: ::: :: :::: :::::@::::@::#:::
|              @::: :::: ::::::@:@ :@:::: @: ::: :: :::: :::::@::::@::#:::
|              @::: :::: ::::::@:@ :@:::: @: ::: :: :::: :::::@::::@::#:::
|        ::::@:@::: :::: ::::::@:@ :@:::: @: ::: :: :::: :::::@::::@::#:::
|        ::: @ @::: :::: ::::::@:@ :@:::: @: ::: :: :::: :::::@::::@::#:::
|        ::: @ @::: :::: ::::::@:@ :@:::: @: ::: :: :::: :::::@::::@::#:::
|   :@::::: @ @::: :::: ::::::@:@ :@:::: @: ::: :: :::: :::::@::::@::#:::
| :::@::::: @ @::: :::: ::::::@:@ :@:::: @: ::: :: :::: :::::@::::@::#:::
| :::@::::: @ @::: :::: ::::::@:@ :@:::: @: ::: :: :::: :::::@::::@::#:::
0 +----------------------------------------------------------------------->Mi
0                                                                     796.8

Number of snapshots: 81
Detailed snapshots: [5, 11, 13, 27, 30, 32, 38, 58, 68, 73 (peak)]
```

Listing 4: Kiểm tra sử dụng bộ nhớ khi sử dụng QuickSort kèm theo Framework.

```
--------------------------------------------------------------------------------
Command:            ./main 5 2
Massif arguments:   --stacks=yes --massif-out-file=Reports/ANALYSIS_DIR/checkmemory
 /MergeSort_Result.out
ms_print arguments: Reports/ANALYSIS_DIR/checkmemory/MergeSort_Result.out
--------------------------------------------------------------------------------


MB
1.071^                                                                  :::
|                                                                      #:::
|                                                                      #:::
|                                                                      #:::
|                                                                      #:::
|              @                                                       #:::
|              @                                                       #:::
|              @                                                       #:::
|              @                                                       #:::
|              @                                                       #:::
|              @::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
|              @::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
```

```
|             @::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
|             @::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
|       ::::::@::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
|       ::::::@::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
|       ::::::@::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
|   ::::::::::@::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
| ::: ::::::::@::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
|:: : ::::::::@::@:::@::@:::::::::::::::::::::@:::::@::::::@:::::@:::#:::
0 +----------------------------------------------------------------------->Mi
0                                                                     787.9

Number of snapshots: 96
Detailed snapshots: [13, 17, 21, 25, 52, 62, 72, 82, 90 (peak)]
```

Listing 5: Kiểm tra sử dụng bộ nhớ khi sử dụng MergeSort.

```
--------------------------------------------------------------------------------
Command:            ./main 5 2
Massif arguments:   --stacks=yes --massif-out-file=Reports/ANALYSIS_DIR/checkmemory
 /MergeSort_Convulation_Result.out
ms_print arguments: Reports/ANALYSIS_DIR/checkmemory/MergeSort_Convulation_Result.
 out
--------------------------------------------------------------------------------


MB
1.134^                                                                  #
|                                                                     @@#
|                                                                   @@@#:::
|                                                                   @@@#:::
|                                                                   @@@#:::
|                                                                   @@@#:::
|            @                                                      @@@#:::
|            @                                                      @@@#:::
|            @                                                      @@@#:::
|            @                                                      @@@#:::
|            @:::::::::::::::@:@@:@:::::@::::::::::::::::@::::@::@@@#:::
|            @::: ::: :::::: @:@ :@:: ::@:::: ::: ::: ::::@::::@::@@@#:::
|            @::: ::: :::::: @:@ :@:: ::@:::: ::: ::: ::::@::::@::@@@#:::
|            @::: ::: :::::: @:@ :@:: ::@:::: ::: ::: ::::@::::@::@@@#:::
|            @::: ::: :::::: @:@ :@:: ::@:::: ::: ::: ::::@::::@::@@@#:::
|       :::@:@::: ::: :::::: @:@ :@:: ::@:::: ::: ::: ::::@::::@::@@@#:::
|       :::@:@::: ::: :::::: @:@ :@:: ::@:::: ::: ::: ::::@::::@::@@@#:::
|   :@:::::@:@::: ::: :::::: @:@ :@:: ::@:::: ::: ::: ::::@::::@::@@@#:::
| :::@:::::@:@::: ::: :::::: @:@ :@:: ::@:::: ::: ::: ::::@::::@::@@@#:::
| :::@:::::@:@::: ::: :::::: @:@ :@:: ::@:::: ::: ::: ::::@::::@::@@@#:::
0 +----------------------------------------------------------------------->Mi
0                                                                     831.5

Number of snapshots: 88
```

```
Detailed snapshots: [5, 11, 13, 27, 30, 32, 38, 58, 68, 73, 76, 78, 81 (peak)]
```

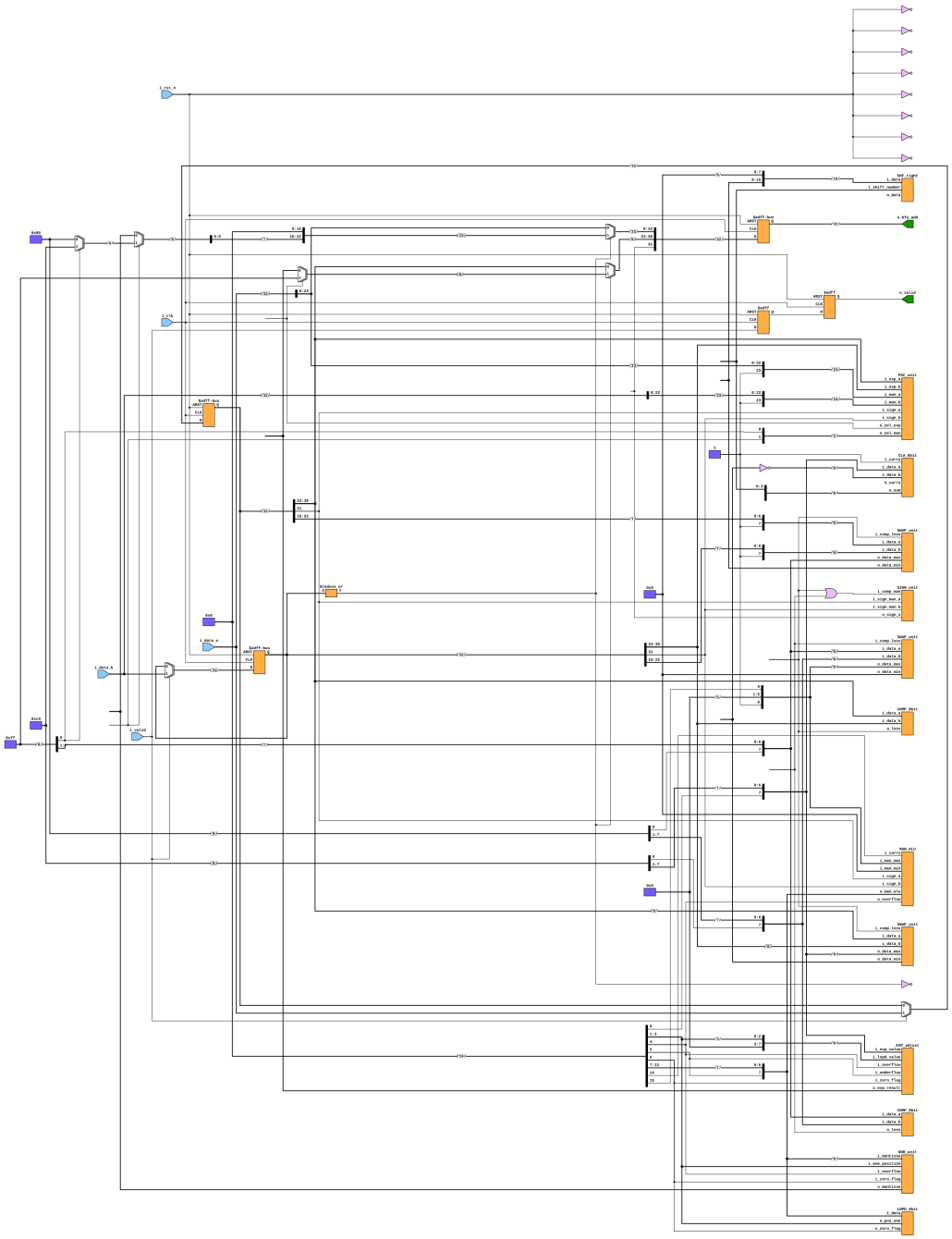
Listing 6: Kiểm tra sử dụng bộ nhớ khi sử dụng MergeSort kèm theo Framework.

Chapter 3. System Level Test

Chapter 4. RTL Implementation

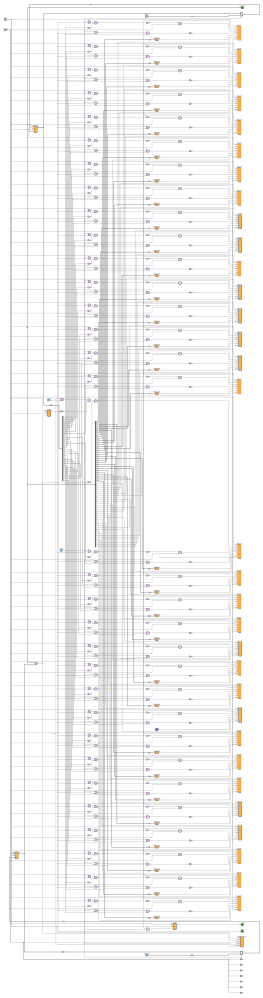
4.1 Bộ thực hiện tính giá trị trung bình của mảng (PACM_UNIT)

4.1.1 Bộ tính cộng số thực (BFP16_add)

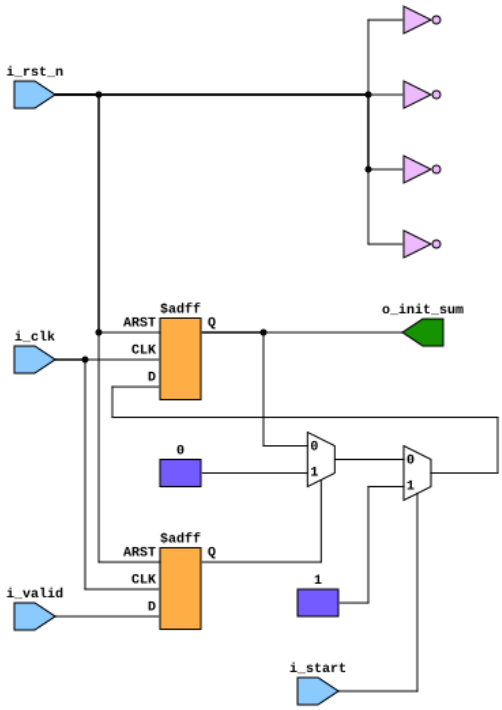


Hình 1: Bộ cộng số thực.

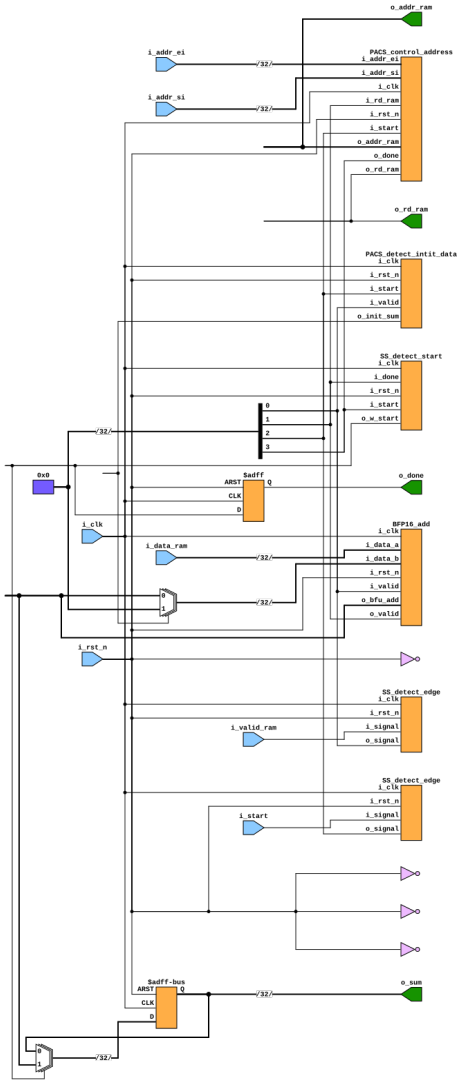
4.1.2 Bộ tính cộng tích lũy của mảng (PA_Cal_Sum)



Hình 2: Bộ điều khiển dữ liệu đọc với Memory Unit.

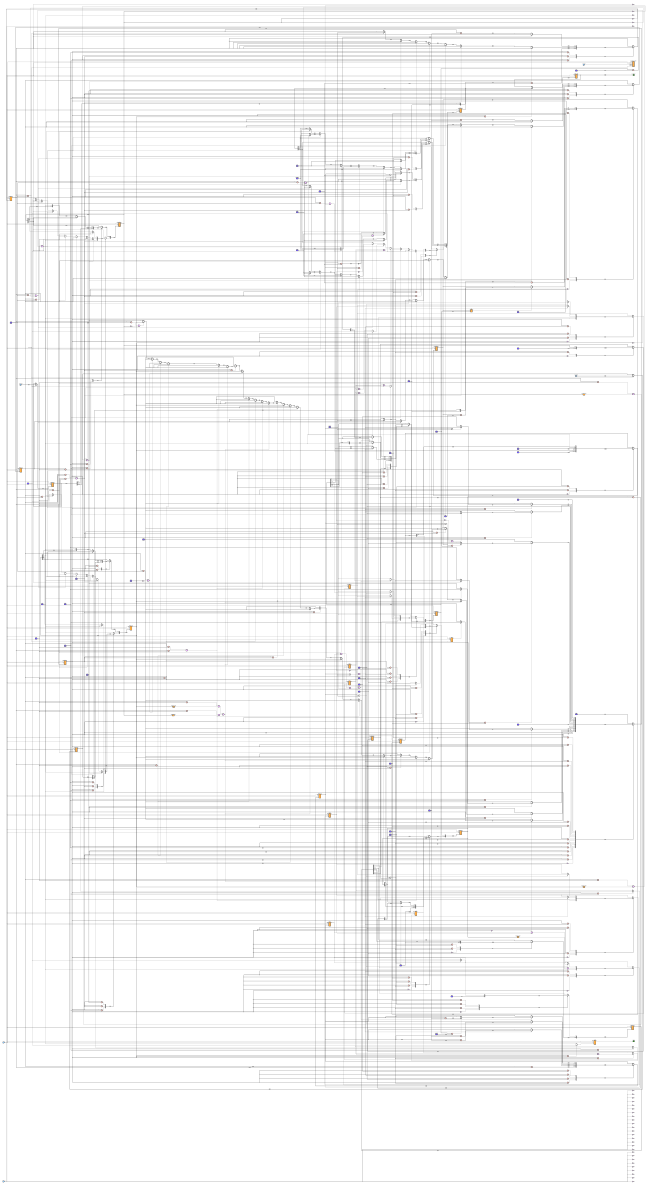


Hình 3: Bộ phát hiện tính hiệu bắt đầu.



Hình 4: Bộ tính cộng số thực (BFP16_add).

4.1.3 Bộ tính chia số thực (FP32_div)



Hình 5: Bộ chia số thực.

4.2 Bộ thực hiện partition lại mảng theo giá trị trung bình (PAMEM_UNIT)

4.3 Bộ thực hiện đệ quy cho giải thuật (DI_UNIT)

Chapter 5. Verification

Tài liệu

[1] S. Shirvani Moghaddam and K. Shirvani Moghaddam, "A general framework for sorting large data sets using independent subarrays of approximately equal length," *IEEE Access*, vol. 10, pp. 11584–11607, 2022.